

CS301 Complete Mid-Term Guide

All Topics from Handout — Lectures 01 to 18 | Simple English + Roman Urdu Hints

18 Lectures Covered

Paper Reviews Included

From Official Handout

CONTENTS — TABLE OF TOPICS

Lec 01 Data Structures + Arrays
Lec 02 List Methods (add/remove/find)
Lec 03 Linked List in Memory + C++
Lec 04 Doubly + Circular + Josephus
Lec 05 ADT + Stack + Array Stack
Lec 06 Linked List Stack + Infix/Postfix
Lec 07 Postfix Evaluation + Conversion
Lec 08 Templates + Function Call Stack
Lec 09 Memory Organization + Queue Intro
Lec 10 Queue Operations + Circular Array
Lec 11 Priority Queue + Binary Tree Terms
Lec 12 BST Insert + Duplicate Search
Lec 13 BST Traversals (In/Pre/Post)
Lec 14 Recursive + Non-Recursive Traversal
Lec 15 Level-Order + BST Delete (3 Cases)
Lec 16 BST Delete Code + BST Class
Lec 17 Reference Variables + & Operator
Lec 18 const Keyword + Transient Objects

LECTURES 01 - 04 — LINKED LISTS

Lec 01 Introduction to Data Structures + Array + List

•What is a Data Structure?

A data structure is a way of organizing data inside a computer so programs run faster and use less memory. When data is large, the right data structure can make a program 100x faster — without buying new hardware.

Hint: *Faster CPU khareedna solution nahi — sahi data structure choose karna solution hai.*

•How to Select a Data Structure

Three things to consider: (1) Space — how much memory it uses, (2) Time — how fast operations run, (3) Programming effort — how hard it is to code. No structure is perfect in all three. You pick based on what matters most for your problem.

Hint: *Har data structure mein trade-off hota hai — koi ek cheez fast, doosri slow. Yeh samajhna zaruri hai.*

•Array

Stores elements in a continuous block of memory. Fixed size. You can access any element instantly using its index ($O(1)$). Problem: inserting or deleting in the middle requires shifting all elements after it — slow ($O(n)$).

```
int x[6];
for(int j = 0; j < 6; j++)
    x[j] = 2 * j;    // stores: 0, 2, 4, 6, 8, 10
```

Hint: Array = fast access (index se seedha), slow insert/delete (shifting karni padti hai).

•List Data Structure

Unlike array, a list does not need continuous memory and has no fixed size. Elements are connected using pointers. You can grow or shrink it at runtime. Insertion and deletion are easier than arrays.

Hint: Array = fixed size, continuous memory. List = flexible size, scattered memory. Yeh fundamental farq yaad rakho.

data structure array list $O(1)$ access $O(n)$ insert

Lec 02 List Methods — add(), next(), remove(), find()

•add() Method

Inserts a new element at the current position. A new node is created, its next pointer is set to the next existing node, and the previous node's pointer is updated to point to the new node.

Hint: Pointer update ki order important hai — pehle naye node ka next set karo, phir purane node ka next update karo. Ulta kiya toh data lost.

•next() Method

Moves the current pointer to the next node. Used to walk through (traverse) the list one node at a time.

Hint: next() = ek qadam aage badho. current pointer move hota hai, kuch delete ya add nahi hota.

•remove() Method

Removes the current node. The previous node's pointer is updated to skip over the removed node and point to the node after it. If memory was dynamically allocated, it must be freed.

```
void remove() {
    lastCurrentNode->setNext(currentNode->getNext()); // step 1
    delete currentNode;                               // step 2
    currentNode = lastCurrentNode;                     // step 3
    size--;                                             // step 4
}
```

Hint: Remove ke 4 steps yaad karo: (1) pointers fix karo, (2) node delete karo, (3) current ko piche lao, (4) size ghataao.

•find() Method

Searches the whole list for a specific value by checking each node one by one from the beginning. Stops when found or list ends. Worst case = $O(n)$.

Hint: find() = puri list check karna. 1 million nodes hain toh 1 million comparisons — isliye BST better hai searching ke liye.

•Analysis of Array vs List

Array: add to middle = $O(n)$ because of shifting. Find = $O(n)$. Linked List: add = $O(1)$ if you have the pointer. Find = $O(n)$. Both are $O(n)$ for search, but list insert is faster once you're at the right position.

add next remove find traversal

Lec 03 Linked List in Computer Memory + C++ Code

•How Linked List Lives in Memory

Array elements sit in one continuous block. Linked list nodes can be anywhere in memory — they are only connected by pointers. The head pointer holds the address of the first node. Each node holds data and the address (pointer) of the next node.

Hint: Sochlo: Array = ghar row mein. Linked list = ghar alag alag jagah, ek dusre ke paas address (pointer) hai jaise directions.

•Node Structure in C++

Each node has two parts: the actual data, and a pointer to the next node. This is the basic building block of every linked list.

```
struct Node {
    int data;          // value stored
    Node* next;        // address of next node
};
```

Hint: Har node = data + next pointer. Bas yeh do cheezein. Itna simple hai.

•Linked List Operations in C++

For inserting at start, end, or middle — you must update pointers carefully in the correct order. One wrong pointer breaks the entire chain and loses access to the rest of the list.

```
// Insert new node at beginning
Node* newNode = new Node();
newNode->data = value;
newNode->next = head;    // new node points to old head
head = newNode;         // head now points to new node
```

Hint: Insert at beginning = sirf 2 pointer assignments. Sequence matter karti hai — pehle newNode ka next lagao, phir head update karo.

[memory](#) [head pointer](#) [nodes](#) [C++ struct](#)

Lec 04 Doubly Linked List + Circular List + Josephus Problem

•Doubly Linked List

Each node has two pointers: next (forward) and prev (backward). You can traverse in both directions. Trade-off: each node uses more memory (2 pointers instead of 1).

```
struct Node {
    int data;
    Node* next;    // forward pointer
    Node* prev;    // backward pointer
};
```

Hint: Doubly = 2 pointers per node. Faida: pichhe bhi ja sako. Nuksan: zyada memory.

•Array vs Linked List — Final Comparison

Array: $O(1)$ random access (jump to any index directly). $O(n)$ insert/delete (shifting needed). Linked list: $O(n)$ to

reach nth element (must walk). $O(1)$ insert/delete if pointer is already there.

Hint: Array = fast access, slow insert. Linked list = slow access, fast insert. Yeh comparison MCQ mein zaroor aata hai.

•Circular Linked List

The last node's next pointer points back to the first node instead of NULL. This creates a loop. You can traverse the list endlessly without hitting NULL.

Hint: Circular = last node ka next, first node par point karta hai. Koi NULL nahi hota end mein.

•Josephus Problem

10 people in a circle. Every 3rd person is eliminated. Last one standing is the leader. A circular linked list models this perfectly — keep removing every 3rd node using a loop until 1 node remains.

Hint: Josephus = circular linked list ka most famous real-world example. MCQ mein "circular list ka use case" poocha jaaye toh yahi answer hai.

doubly linked prev pointer circular list josephus

LECTURES 05 - 09 — STACK, ADT, EXPRESSIONS, MEMORY

Lec 05 Abstract Data Type (ADT) + Stack + Stack using Array ▼

•Abstract Data Type (ADT)

An ADT defines what operations a data structure supports — but hides how they are implemented. The user sees only the interface (what it does), not the internal code (how it does it). Stack, Queue, and BST are all ADTs.

Hint: ADT = TV remote. Aap "volume up" press karte ho — andar circuit kaise kaam karta hai, aapko nahi pata. Interface public, internals hidden.

•Stack — What Is It?

A LIFO structure — Last In, First Out. The most recently added item is the first one removed. All operations (push and pop) happen only at the top.

LIFO = Last In, First Out. Stack of plates — top plate is always taken first. Cannot pull from the bottom.

•Stack Operations

push(x): add x to top. pop(): remove and return top element. top(): peek at top without removing. isEmpty(): check if stack is empty. isFull(): check if stack is full (array implementation only).

Hint: isEmpty() = stack ka own property hai. isFull() = sirf array implementation mein hota hai — array ki limitation ki wajah se.

•Stack using Array

Use an array and an integer variable "top" (starts at -1). Push: increment top, store element at top. Pop: return element at top, decrement top.

```
push(x): top++; arr[top] = x;
pop():   value = arr[top]; top--;
top():   return arr[top];
```

Hint: Stack overflow = array full tha aur push kiya. Stack underflow = stack empty tha aur pop kiya. Dono error

cases MCQ mein aate hain.

ADT LIFO push pop top overflow underflow

Lec 06 Stack using Linked List + Operator Precedence + Infix to Postfix

•Stack using Linked List

Build a stack using a linked list instead of an array. Advantage: no fixed size — grows dynamically, no overflow. Disadvantage: each node uses extra memory for the pointer.

Hint: Array stack = fast, fixed size. Linked list stack = unlimited size, thodi zyada memory. Dono valid hain — context par depend karta hai.

•Infix, Postfix, Prefix

Infix: operator between operands — $A+B$. How humans write math. Postfix: operator after operands — $AB+$. What computers prefer. Prefix: operator before operands — $+AB$.

Postfix needs no parentheses or precedence rules during evaluation — just a stack. That is why computers use it internally.

Infix		Postfix
$A+B$	->	$AB+$
$A+B*C$	->	$ABC*+$
$(A+B)*C$	->	$AB+C*$

Hint: Postfix mein brackets ki zarurat nahi — position of operators sab kuch batata hai. Computer ke liye easy.

•Operator Precedence

Priority order (highest to lowest): $^$ (exponent) $>$ $*$ $/$ $>$ $+$ $-$. When converting infix to postfix, this determines which operators to pop from the stack first.

Hint: Precedence yaad karo: $^$ sabse upar, phir $*$ / $/$, phir $+$ / $-$. Parentheses is order ko override kar sakte hain.

•Infix to Postfix — Basic Algorithm

Read left to right. Operand: write to output. Operator: pop higher/equal precedence operators from stack to output, then push new operator. '(' : push. ')' : pop until '(' found, discard both. End: pop all remaining stack to output.

Hint: Key rule: operator aaya toh stack se pehle same/higher precedence wale nikalo output mein, PHIR naya push karo.

linked list stack infix postfix prefix precedence

Lec 07 Evaluating Postfix + Full Infix to Postfix with Table

•Evaluating a Postfix Expression

Read left to right. Number: push onto stack. Operator: pop two numbers, apply the operator (second popped is left operand, first popped is right operand), push result. Final answer = last item on stack.

Expression:	4	3	5	*	+
Push 4					Stack: [4]
Push 3					Stack: [4, 3]
Push 5					Stack: [4, 3, 5]
* -> pop 5,3 -> 3*5=15 -> push 15					Stack: [4, 15]

```
+ -> pop 15,4 -> 4+15=19 -> push 19 Answer: 19
```

Hint: Pop order: pehla pop = right operand, doosra pop = left operand. a op b. Ulta mat karna warna galat answer aayega.

•Infix to Postfix — Table Method (Exam Format)

This is the exact format that appeared in multiple papers. Three columns: Symbol (input one by one), Stack (current stack state), Postfix (output so far).

Example: Convert (A+B)*C

Symbol	Stack	Postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
)	(empty)	AB+
*	*	AB+
C	*	AB+C
end	(empty)	AB+C*

Hint: Table mein ek ek step likho. ')' aaya toh '(' tak sab pop karo output mein. Brackets khud output mein nahi aate.

postfix evaluation

stack

table method

exam format

Lec 08 C++ Templates + Function Call Stack + Activation Record

•C++ Templates

A template lets you write one class or function that works for any data type. Without templates, you would need a separate Stack class for int, float, string, etc. With templates, write once, use for all.

```
template <class T>
class Stack {
    T data[100]; // works for any type T
    int top;
    void push(T item) { data[++top] = item; }
    T pop() { return data[top--]; }
};
```

Hint: Template = ek mold, alag alag cheezein banao (int stack, float stack, string stack) — code repeat nahi karna.

•Function Call Stack

Every time a function is called, the computer pushes an "activation record" (stack frame) onto the call stack. This frame contains: function parameters, local variables, and return address. When the function returns, the frame is popped and execution continues from the return address.

Hint: Function call = stack par push. Function return = stack se pop. Yeh mechanism sab languages mein same hai — C++, Java, FORTRAN sab.

•Recursion and the Call Stack

Recursive functions use the call stack to save their state. Each recursive call pushes a new frame. When the

base case is reached, frames are popped one by one as each call returns.

Hint: Recursion bahut deep ho jaaye toh stack bhar jaata hai — yahi "Stack Overflow" hai (us famous website ka naam yahan se aaya).

templates

generic programming

function call

activation record

recursion

Lec 09 Memory Organization + Queue Introduction + Queue using Array

•Memory Organization

When a program runs, OS divides memory into 4 sections: (1) Code — compiled program instructions. (2) Static Data — global variables. (3) Stack — for function calls (grows and shrinks). (4) Heap — for dynamic memory (new/malloc).

Memory Layout:

```
+-----+
| Code Section | (program instructions)
+-----+
| Static Data  | (global variables)
+-----+
| Stack        | (function calls, grows down)
+-----+
| Heap         | (new/malloc, grows up)
+-----+
```

Hint: Stack aur Heap alag hain — Stack = function calls, Heap = new/malloc. Dono ko mix mat karna.

•Stack Layout During Function Call

When function A calls B, a new activation record for B is pushed onto the stack. It holds B's parameters, local variables, and the return address (where to go when B finishes). When B returns, its frame is popped and A continues.

Hint: Activation record = function ka sara saman. Function return hone par saman uth jaata hai stack se.

memory layout

code section

static data

stack

heap

LECTURES 10 - 11 — QUEUE AND BINARY TREE BASICS

Lec 10 Queue — Operations, Circular Array, Use Cases

•What is a Queue?

A FIFO structure — First In, First Out. Items enter from the rear and leave from the front. The first item added is the first one removed.

Stack = LIFO. Queue = FIFO. Yeh opposite hain. MCQ mein sabse common mistake yahi hai — mix karna.

•Queue Operations

enqueue(x): add x at the rear. dequeue(): remove and return from front. front(): peek at front without removing. isEmpty(): check if empty. Queue does NOT have a "pop" operation — that belongs to Stack.

Hint: Queue ka "pop" nahi hota — yeh Stack ka operation hai. Queue mein "dequeue" hota hai. Yeh exact MCQ papers mein aaya tha.

•Queue using Array + Circular Array

Two pointers: front and rear. Enqueue: rear++ then add. Dequeue: read from front, then front++. Problem: as elements are dequeued, space at the start is wasted and cannot be reused. Solution: Circular array — when rear reaches the end, wrap it back to index 0.

```
// Circular array trick
rear = (rear + 1) % arraySize; // wrap around
```

Hint: Circular array formula: $\text{rear} = (\text{rear} + 1) \% \text{size}$. Isse starting space reuse hota hai aur array kabhi waste nahi hoti.

•Use of Queues

Queues are used wherever order matters: OS job scheduling, print spooler, bank simulations, network packet handling. Anything where "first come, first served" applies.

Hint: Queue = bank line, printer queue, CPU scheduling. FIFO = fair order — jo pehle aaya pehle serve hoga.

FIFO enqueue dequeue front rear circular array

Lec 11 Priority Queue + Tree + Binary Tree + Terminology

•Priority Queue

A variation of queue where each element has a priority value. The element with the highest priority is dequeued first — regardless of when it was added. Not strictly FIFO.

Hint: Hospital emergency = priority queue. Serious patient pehle serve hoga chahe baad mein aaya ho. Priority = urgency.

•Tree Data Structure

A hierarchical (non-linear) structure. One root node at the top. Each node can have child nodes below it. Looks like an upside-down tree. Unlike lists and arrays (linear), trees branch out.

•Binary Tree

A tree where every node has at most 2 children: left child and right child. No node can have more than 2 children.

•Key Binary Tree Terminology

Root: topmost node, no parent. Leaf: node with no children. Parent/Child: directly connected nodes above/below. Height: number of edges from root to deepest leaf. Level: root = level 0; its children = level 1, etc. Depth: how far a node is from the root.

Hint: Root se neeche level count karo (0, 1, 2...). Leaf = koi child nahi. Height = deepest level. Yeh teeno terms alag hain.

•Strictly Binary Tree

Every node has either 0 or exactly 2 children. No node with just 1 child is allowed.

Hint: Strictly binary = 0 ya 2 children, kabhi 1 nahi. Complete binary = sab levels full (ya last level left se fill). Dono alag definitions hain.

•Complete Binary Tree + Depth Formula

A complete binary tree fills all levels from left to right. If n = total nodes, depth $d = \log_2(n+1) - 1$. For $n = 100,000$: $d = \log_2(100001) - 1 \approx 16$.

Formula: $d = \log_2(n + 1) - 1$


```
n = 100,000:  
d = log2(100001) - 1 = 16.6 - 1 ≈ 16
```

Hint: Handout mein $d=20$ diya hai lekin formula se $d=16$ aata hai. Calculation show karo — formula sahi hai.

priority queue tree binary tree root leaf height depth formula

LECTURES 12 - 16 — BINARY SEARCH TREE (BST)

Lec 12 BST Operations + Insert + Searching for Duplicates

•BST Rule

For every node in a BST: all values in its left subtree are smaller, all values in its right subtree are larger. This rule must hold for every single node in the tree — not just the root.

```
      14  
     /  \  
    4    15  
   /  \  
  3    9    18  
Left of 14: {4,3,9} -- all less than 14 ✓  
Right of 14: {15,18} -- all greater than 14 ✓
```

Hint: BST rule = left chhota, right bara. Har node par apply hota hai — sirf root par nahi. Yeh sabse important rule hai.

•Insert in BST

Start from root. If new value is smaller, go left; if larger, go right. Repeat at each node until you hit NULL — insert the new node there.

Hint: Insert = compare and go left/right until NULL milta hai. NULL spot = yahan naya node lagao.

•Searching (Duplicate Check)

Same logic as insert. At each node, compare target with current value. Go left if smaller, right if larger. If match found = duplicate. If NULL reached = not in tree. Average time: $O(\log n)$. Worst case (skewed tree): $O(n)$.

Hint: BST mein search = har step mein aadha tree eliminate hota hai. 1 million nodes = sirf ~20 comparisons. Linked list mein same 1 million steps lagte.

•Applications of Binary Tree

Expression trees (store mathematical expressions), search trees (BST for fast lookup), decision trees, file system structures, and more.

BST rule insert search $O(\log n)$ duplicate

Lec 13 BST Traversals — Inorder, Preorder, Postorder + C++ Code

•Three Traversal Methods

Inorder (Left, Root, Right): for BST, always gives sorted output. Most useful. Preorder (Root, Left, Right): root first — useful for copying/printing a tree. Postorder (Left, Right, Root): root last — useful for deleting a tree, evaluating expression trees.

```
Tree:      4
          / \
         2  6
        / \
       1  3
```

Inorder: 1, 2, 3, 4, 6 ← SORTED always

Preorder: 4, 2, 1, 3, 6

Postorder: 1, 3, 2, 6, 4

Hint: Inorder of BST = always sorted. Yeh guaranteed property hai — baar baar exam mein aati hai. Yaad karo.

•C++ Recursive Code

All three traversals are naturally recursive — just 4-5 lines each. Only the order of the three steps changes.

```
void inorder(Node* n) {
    if(n == NULL) return;
    inorder(n->left);    // Left
    cout << n->data;     // Root
    inorder(n->right);   // Right
}

void preorder(Node* n) {
    if(n == NULL) return;
    cout << n->data;     // Root first
    preorder(n->left);
    preorder(n->right);
}
```

Hint: In = L-Root-R, Pre = Root-L-R, Post = L-R-Root. Sirf order yaad karo — code structure same rahega.

inorder

preorder

postorder

sorted output

recursion

Lec 14 Recursive Traversal Trace + Non-Recursive Traversal

•How Recursive Calls Work on the Stack

When a recursive traversal function calls itself, a new stack frame is pushed with the current node. When it returns (hits NULL or finishes), that frame is popped. The call stack keeps track of where to go back to.

Hint: Recursion trace mein: function call = stack push, return = stack pop. Yeh trace draw karna exam mein aa sakta hai.

•Preorder Recursion Trace Example

For preorder on a tree: root is printed first, then left subtree is fully explored (all recursive calls), then right subtree. The stack keeps the "return to" addresses so we always come back and finish the right side.

Hint: Trace mein dikhao: kaunsa node print hua, stack mein kya tha us waqt. Ek ek step likhna padta hai.

•Non-Recursive Traversal

Instead of using system recursion, maintain your own explicit stack. Push nodes manually, pop them to process. This gives you more control and avoids deep recursion issues.

```
// Non-recursive inorder using explicit stack
stack<Node*> s;
Node* curr = root;
while(curr != NULL || !s.empty()) {
    while(curr != NULL) { s.push(curr); curr = curr->left; }
    curr = s.top(); s.pop();
    cout << curr->data << " ";
    curr = curr->right;
}
```

```

curr = s.top(); s.pop();
cout << curr->data;
curr = curr->right;
}

```

Hint: Non-recursive = apna stack banao manually. Same result, different implementation. Deep trees ke liye better hota hai.

recursive trace call stack non-recursive explicit stack

Lec 15 Level-Order Traversal + BST with Strings + Delete Node (3 Cases)

•Level-Order Traversal (BFS)

Visit tree level by level — root first (level 0), then all level 1 nodes, then all level 2 nodes, etc. Uses a Queue, not a Stack. Enqueue root, then for each dequeued node, enqueue its children.

```

Queue: [14]
Dequeue 14, print 14, enqueue 4 and 15
Queue: [4, 15]
Dequeue 4, print 4, enqueue 3 and 9
Queue: [15, 3, 9] ... and so on

Output: 14, 4, 15, 3, 9, 18

```

Level-order = Queue. Normal traversals (in/pre/post) = Stack (via recursion). Yeh farq MCQ mein aata hai.

•BST with Strings

BST works with any comparable data, not just numbers. For strings, comparison is alphabetical. "apple" < "banana" so apple goes left. All BST rules apply exactly the same.

Hint: String BST = same rules, comparison alphabetical hoti hai. Koi alag rule nahi.

•Deleting a Node from BST — 3 Cases

Case 1 — Leaf node (0 children): Simplest. Just remove it. Set parent's pointer to NULL. Free memory.

Case 2 — One child: Remove the node, connect parent directly to that one child. Skip over the deleted node.

Case 3 — Two children (hardest): Find the inorder successor = the smallest value in the right subtree (go right once, then keep going left until NULL). Copy its value into the node to be deleted. Then delete the inorder successor (which is always Case 1 or Case 2).

```

Delete 14 from:      Result:
    14                15
   /  \              /  \
  4    15            4    18
 /  \   \           /  \   \
3   9   18         3   9   20
                  \
                  20

```

Inorder successor of 14 = 15 (right, then go left = no left, so 15)
Copy 15 to root, delete original 15 node.

Hint: Case 3 shortcut: inorder successor = right subtree ka sabse left (chhota) node. Uski value copy karo, phir use delete karo (Case 1 ya 2 hoga woh).

level-order queue BST delete 3 cases inorder successor

•C++ Code for BST Remove

The remove function first searches for the target. Then checks the case (0, 1, or 2 children) and adjusts pointers. Important property: the inorder successor always has no left child — because if it had one, that child would be smaller, and thus it would be the leftmost node, not the inorder successor.

Hint: Inorder successor ka guaranteed property: left child nahi hota. Yeh deletion simplify karta hai — always Case 1 ya 2 hoga.

•BST as a C++ Class

Wrap the entire BST in a class. Public interface: insert(), remove(), find(). Private: the root pointer and all recursive helper functions. This is the ADT concept applied — users call public methods without knowing the tree structure inside.

```
class BST {
private:
    Node* root;
    void insertHelper(Node*&, int);
    void removeHelper(Node*&, int);
public:
    void insert(int x) { insertHelper(root, x); }
    void remove(int x) { removeHelper(root, x); }
    bool find(int x);
};
```

Hint: Class = ADT ka C++ implementation. Public = user ka interface. Private = andar ki machinery. Yeh encapsulation hai.

BST class remove code encapsulation public/private

•Reference Variable — What Is It?

A reference variable is a second name (alias) for an existing variable. Both names refer to the same memory location. Change one, the other automatically changes too.

```
int x = 10;
int& ref = x; // ref is an alias for x
ref = 20;     // x is now also 20
cout << x;    // prints 20
cout << ref;  // also prints 20
```

Hint: Reference = doosra naam, same memory. Ek badla dono badle. Yeh pointer se alag hai — no * needed to access value.

•The & Symbol — 3 Different Uses

(1) Address operator: &x returns the memory address of x. (2) Reference declaration: int& r = x means r is an alias for x. (3) Bitwise AND: a & b performs bitwise AND operation.

Hint: & ka teen jagah alag matlab: &x = address, int& r = reference, a&b = bitwise AND. Context dekh ke pehchano — MCQ mein aata hai.

•Call by Value vs Reference vs Pointer

Call by Value: a copy is passed. Original is not changed. Call by Reference: the original variable is passed directly using &. Changes inside function affect the original. Call by Pointer: pointer to variable is passed. Dereference to change original.

```
void byValue(int x)    { x = 99; } // original unchanged
void byRef(int& x)    { x = 99; } // original CHANGED
void byPtr(int* x)    { *x = 99; } // original CHANGED

int n = 5;
byValue(n); // n still 5
byRef(n);   // n is now 99
byPtr(&n);  // n is now 99
```

Hint: Call by value = copy jaati hai, original safe. Call by reference = original jaata hai, change hota hai. Dono ka farq yaad rakho.

reference variable

alias

& operator

call by value

call by reference

call by pointer

Lec 18 const Keyword + Transient Objects + Dangling References

•const Keyword

const locks a variable — it cannot be changed after being set. In function parameters, const means "I promise not to modify this value." The compiler enforces this — any accidental modification causes a compile error.

```
void insert(const int& x) {
    // x is passed by reference (no copy made — fast)
    // but x cannot be modified (safe)
    // x = 99; // this would cause a compile ERROR
}
```

Hint: const + reference = best combo: no copy (fast) + no accidental change (safe). Yahi BST insert/find mein use hota hai.

•Transient Objects and Dangling References

Local variables are created on the call stack and destroyed when the function returns. If you store a reference to a local variable inside a data structure, and the function returns, that reference now points to destroyed (invalid) memory. This is a "dangling reference" — very serious bug.

```
// DANGEROUS — never do this
int& badFunction() {
    int local = 10;
    return local; // local is destroyed after return!
}                // returned reference is now dangling!
```

Hint: Local variable ka reference data structure mein store mat karo — function return hone ke baad woh memory invalid ho jaati hai. Dangling reference = undefined behavior.

•Stack Usage Across Languages

The function call stack mechanism is universal — same concept in Java, FORTRAN, Pascal, and C++. Java passes native types (int, float) by value and objects by reference. FORTRAN passes everything by reference. Pascal can do both like C++.

Hint: Stack mechanism = hardware level concept. Sab languages use karte hain — sirf syntax alag hai.

const

const reference

transient object

dangling reference

local variable

MASTER CHEAT SHEET — SAB KUCH EK JAGAH

Topic	Key Point
Stack vs Queue	Stack = LIFO (last in first out). Queue = FIFO (first in first out). Pop = Stack. Dequeue = Queue.
BST Rule	Left < Root < Right. Every node follows this — not just root.
Inorder of BST	Always gives sorted output. Guaranteed. L → Root → R.
Level-order	Uses Queue. Top to bottom, left to right. Not a stack.
BST Delete Case 3	Inorder successor = right subtree ka leftmost (smallest) node. Copy its value, delete it.
Postfix evaluation	Number = push. Operator = pop 2, compute, push result. Second popped = left operand.
Circular array	rear = (rear + 1) % size. Avoids wasted space at front.
Circular linked list	Last node's next = first node (not NULL).
Josephus	Circular list use case. Every 3rd node removed. Last = leader.
const & parameter	Fast (no copy) + safe (no modification). Best practice.
Reference variable	int& r = x means r is alias for x. Same memory. Change one = change both.
Dangling reference	Reference to local variable after function returns = invalid memory = bug.
Stack overflow	Push to full array stack, OR too deep recursion filling call stack.
ADT	Define WHAT it does, hide HOW. Public interface + private implementation.
BST depth formula	$d = \log_2(n+1) - 1$. For $n=100,000 \rightarrow d \approx 16$.
Array vs Linked List	Array = $O(1)$ access, $O(n)$ insert. Linked list = $O(n)$ access, $O(1)$ insert.